



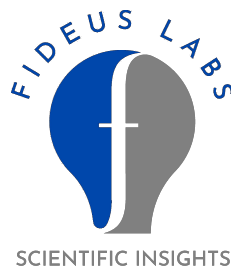
SciPy 2026 · Biological & Medical Sciences

ngff-zarr

A Lean and Kind OME-Zarr Toolkit for Bioimaging

Matt McCormick

Fideus Labs · SciPy Conference, University of Minnesota · July 2026



Bioimaging data is exploding — and locked in silos

Big, multidimensional, proprietary

Modern microscopes and scanners produce terabyte-scale, 5D (t, c, z, y, x) images in dozens of vendor-specific formats.

Not built for how we work now

Monolithic files are hard to stream, share, or open in the cloud — blocking reproducible, collaborative science and FAIR data.

TB+

per acquisition

5D

t, c, z, y, x

dozens

of formats

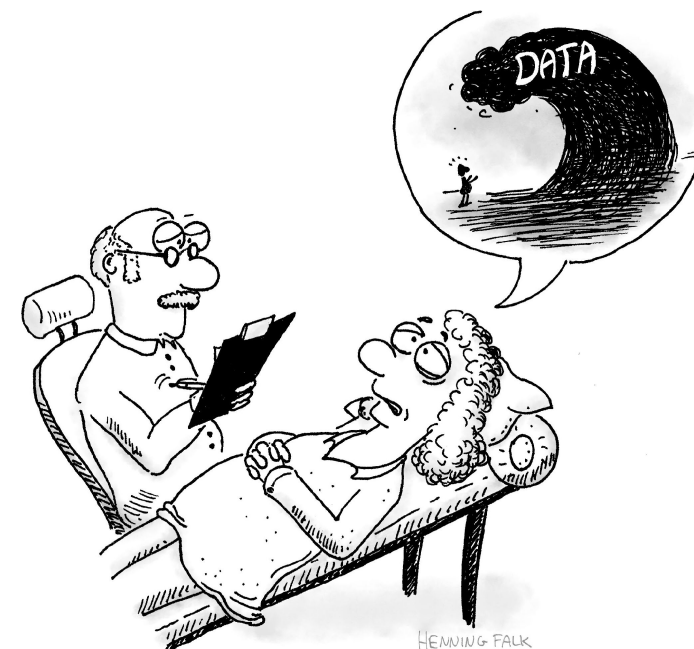


Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)

The community's answer: OME-Zarr

An open, chunked, cloud-native standard for bioimaging — built on Zarr and developed in the open.

Chunked & compressed

Read one region of a huge image without downloading the whole file — the basis of cloud-optimized access.

Multiscale

Each image is a resolution pyramid, enabling responsive viewing and scale-appropriate analysis.

Self-describing

Axes, units, and transforms travel with the pixels in a shared, machine-readable data model.



One store, many viewers — Fiji/MoBIE, napari, Neuroglancer, webKnossos.

Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](#)

Community-developed · [Moore et al. 2021](#) · [Moore et al. 2023](#) · [Lüthi et al. 2025](#)

A spec needs tooling: ngff-zarr

ngff-zarr is a **lean** and **kind** Python implementation of OME-Zarr.

Lean

Minimal dependencies on top of the scientific Python stack — lazy, parallel, and web-ready, with no local filesystem required.

Kind

An interface that maps to how researchers think about their data — including a natural-language interface for AI agents.

github.com/fideus-labs/ngff-zarr · ngff-zarr.readthedocs.io

```
> ngff-zarr -i ./MR-head.nrrd
NGFF OME-Zarr
Multiscales(
  images=[
    NgffImage(
      data=dask.array<rechunk-merge, shape=(130, 256, 256),
      dtype=int16, chunksize=(128, 128, 128), chunktype=numpy.ndarray>,
      dims=('z', 'y', 'x'),
      scale={'z': 1.2999954223632812, 'y': 1.0, 'x': 1.0},
      translation={
        'z': 116.78569793701172,
        'y': -133.92860412597656,
        'x': 86.64489746093749
      },
      name='image',
      axes_units=None
    ),
    metadata=Metadata(
      axes=[
        Axis(name='z', type='space', unit=None),
        Axis(name='y', type='space', unit=None),
        Axis(name='x', type='space', unit=None)
      ],
      datasets=[
        Dataset(
          path='scale0/image',
          coordinateTransformations=[
            Scale(
              scale=[1.2999954223632812, 1.0, 1.0],
              type='scale'
            ),
            Translation(
              translation=[
                116.78569793701172,
                -133.92860412597656,
                86.64489746093749
              ],
              type='translation'
            )
          ],
          name='image',
          version='0.4',
          coordinateTransformations=[]
        ),
        scale_factors=[],
        method=<Methods.DASK_IMAGE_GAUSSIAN: 'dask_image_gaussian'>,
        chunks={'z': 128, 'y': 128, 'x': 128}
      ],
      information=
    )
  ]
)
```

`ngff-zarr -i MR-head.nrrd` — the self-describing data model

[Source: ngff-zarr docs](#)

01

The approach

A four-step pipeline that mirrors the OME-Zarr data model.



Four steps: array → image → multiscales → store

array

Any Array API input
NumPy · Dask · CuPy · PyTorch

NgffImage

Attach dims, scale,
translation & units

NgffMultiscales

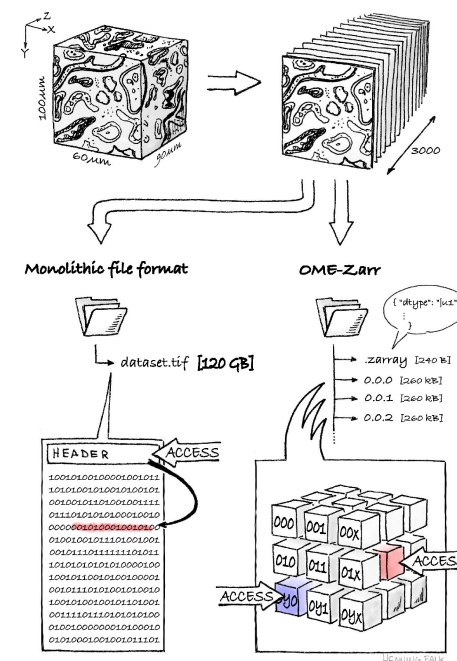
Lazy Dask pyramid with
metadata

OME-Zarr store

Written out-of-core
local or cloud (fsspec)

```
import ngff_zarr as nz

image = nz.to_ngff_image(array, dims=["z", "y", "x"], scale={...})
ms = nz.to_multiscales(image, scale_factors=[2, 4], chunks=64)
nz.to_ome_zarr("output.ome.zarr", ms)
```



Monolithic vs. chunked access

Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)

Lazy Dask graphs process data larger than memory

Everything is a task graph

Each pipeline stage builds a lazy Dask graph. Chunks stream through smoothing, downsampling, and writing — nothing is materialized all at once.

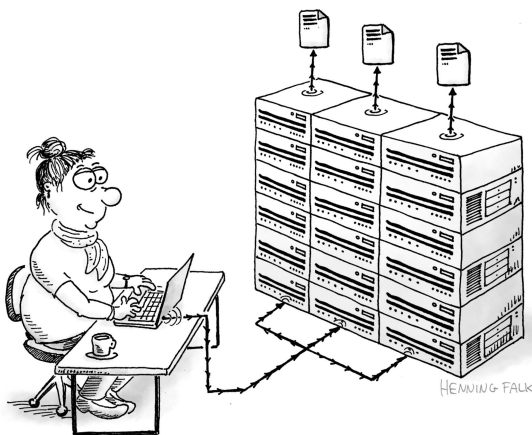
Convert a 100+ GB volume on a laptop

A memory-aware scheduler keeps peak RAM bounded, so extremely large datasets convert on modest hardware.

```
# CLI, bounded to 50 MB of memory
ngff-zarr --memory-target 50M \
  -i LIDCFull.vtk \
  -o LIDCFull.ome.zarr
```

Web-ready by design

Read and write directly to S3, GCS, or Azure via fsspec — no local filesystem required.



Bea computes

Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)

Faithful pyramids: pick the right downscaling method

Smoothing before downsampling avoids aliasing. Label images get dedicated mode-based reductions.

Method	Backend	Image type
ITKWASM_GAUSSIAN (default)	ITK-Wasm · SIMD / opt. CUDA	Intensity
ITKWASM_BIN_SHRINK	ITK-Wasm · SIMD / opt. CUDA	Intensity
ITKWASM_LABEL_IMAGE	ITK-Wasm	Label
ITK_GAUSSIAN / ITK_BIN_SHRINK	Native ITK	Intensity
DASK_IMAGE_GAUSSIAN	dask-image / SciPy	Intensity
DASK_IMAGE_MODE / _NEAREST	dask-image / SciPy	Label

02

Features

From single-file archives to anatomical orientation, HCS, and coordinate transforms.



FEATURES

One lean toolkit, broad real-world coverage

.ozx single-file archives

Package a whole store into one Zip (RFC-9) for easy sharing and archival.

RFC-4 anatomical orientation

LPS/RAS axis metadata for medical & neuroimaging interoperability.

High Content Screening

OME-Zarr plate/well model — navigate by row, column, and field.

Broad format conversion

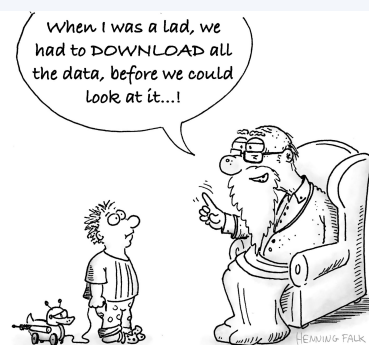
TIFF / OME-TIFF, Leica LIF, and anything ITK, tiff file, or imageio can read.

Zarr v3 sharding

Pack many chunks per file to cut file counts on huge outputs.

Validation & CLI

Spec-compliance checks and a batch CLI mirroring the library.



Qui downloads.

Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](#)

ngff-zarr-mcp: bioimaging through natural language

The **ngff-zarr-mcp** server exposes conversion, inspection, validation & optimization to AI agents via the [Model Context Protocol](#).

convert_images_to_ome_zarr

Convert datasets with configurable parameters

get_ome_zarr_info

Report detailed store structure & metadata

validate_ome_zarr_store

Check compliance against the specification

optimize_ome_zarr_store

Re-encode with new compression / chunking

Researchers just ask:

"Convert LIDCFull.tif to OME-Zarr."

"Find the optimal codec for this data."

"Use sharding to keep files under 20."

"Write a script to convert this folder."

Presented at EMBL Advanced Methods in Bioimage Analysis, 2025

Designing tools for agents, not just APIs

1

Structure beats free text

Clearly typed, well-described parameters make agent calls far more reliable than free-form strings.

2

Map to intent, not internals

A single high-level tool an agent can reason about beats exposing the whole internal API surface.

3

Language lowers the barrier

Letting scientists describe what they want — instead of learning an API — drives adoption of cloud-native, reproducible workflows.

03

What's new & what's next

Initial OME-Zarr 0.6 / RFC-5 support today — and a roadmap to 1.0.



Initial OME-Zarr 0.6 & RFC-5 coordinate transforms

RFC-5 brings first-class coordinate systems and transformations — the centerpiece of the emerging version 0.6.

Named coordinate systems

Describe how multiple images relate — aligned tiles, registered modalities — without resampling pixels.

Rich transformations

Identity, permutation, translation, scale, affine, rotation, sequences, and displacement fields.

0.6 in ngff-zarr today

Coordinate-system handling in multiscale generation, an updated 0.6 schema, and round-trip conversion between 0.5 and 0.6.

Pairs with RFC-4

RFC-5 maps image space to anatomical space; RFC-4 supplies the biological orientation labels.

The road to OME-Zarr 1.0 alpha

Spec

Deeper coordinate-transformation support & tooling to work with transforms

OME-Zarr 1.0 alpha over the coming year

RFC-3 support (more dimensions)

RFC-8 — Collections & Extensibility

Performance & platform

Improved parallelism

Optional GPU support

Fewer dependencies

Ray support for distributed scaling



Navid zooms.

Illustration: Henning Falk. © NumFOCUS. CC BY 4.0

TAKEAWAYS

What to remember from this talk

1

OME-Zarr is the cloud-native standard

Chunked, multiscale, self-describing — and community-driven.

2

ngff-zarr makes it a few lines of Python

array → NgffImage → NgffMultiscales → store, out-of-core via Dask, local or cloud.

3

It goes beyond conversion

.ozx, RFC-4, HCS, sharding, a CLI — and now RFC-5 / OME-Zarr 0.6.

4

AI agents can drive it in natural language

The MCP server lowers the barrier to reproducible, cloud-native workflows.

Thank you

Convert, validate & scale bioimaging data — leanly and kindly.

SOURCE CODE

github.com/fideus-labs/ngff-zarr

DOCUMENTATION

ngff-zarr.readthedocs.io

PYPI and CONDA-FORGE

`pip install ngff-zarr`

MCP SERVER

`uvx ngff-zarr-mcp`

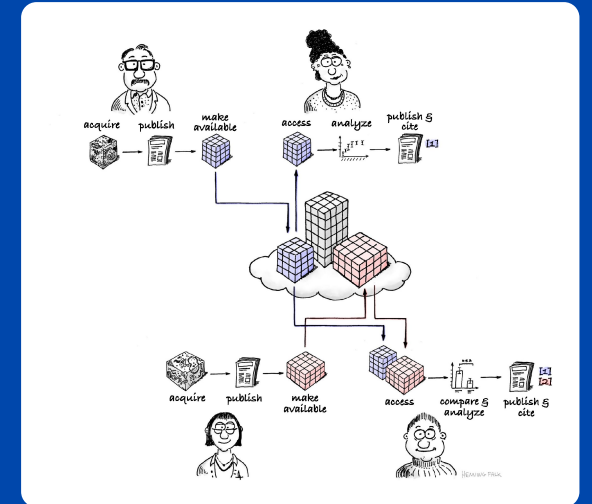


Illustration: Henning Falk, © NumFOCUS, [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)